Real-Time Delta One Risk Dashboard

Project Guide — Architecture, Code & Interview Prep

Author	Théodore Dyrlund
GitHub	github.com/theodoredyrlund/real-time-delta-one-risk-dashboard
Stack	Python · Streamlit · Pandas · yfinance · Docker · AWS
Date	May 2026

1. What Is a Delta One Desk?

A **Delta One product** is a derivative or financial instrument whose price moves approximately one-for-one with the underlying asset — delta = 1, no optionality, no convexity. Examples include ETFs, total return swaps (TRS), equity index futures, CFDs, and basket replication.

Delta One desks make money through **financing spreads**, **dividend arbitrage**, and **basis trading** rather than directional or volatility bets. They need to monitor in real time: PnL by position, gross/net exposure, beta to the market, Value at Risk, and concentration limits.

2. Project Overview

This project simulates the core risk monitoring tool of a Delta One desk. It fetches live market data, manages a simulated ETF portfolio (long/short), computes financial risk metrics, and displays everything in a professional Streamlit dashboard.

Component	Description	Technology
Data Ingestion	Live & historical market prices	yfinance (Yahoo Finance)
Portfolio Engine	Positions, market values, exposures	Python / Pandas
PnL Engine	Per position, total, intraday, history	Python / NumPy
Risk Engine	VaR, beta, vol, drawdown, correlation	Python / SciPy
Database	Persistence of snapshots & history	SQLite → PostgreSQL
Dashboard	Interactive UI with charts & alerts	Streamlit / Plotly
Infra	Containerisation & cloud deployment	Docker / AWS ECS

3. Architecture

3.1 MVP (local)

```
Market Data API (yfinance) ■ ▼ src/market_data.py ■■■ SQLite database (data/dashboard.db) ■ ▼ src/portfolio.py ■■■ src/pnl.py ■■■ src/risk.py (positions/expo) (PnL calcul) (VaR/beta/vol/DD) ■ ▼ app/dashboard.py (Streamlit — localhost:8501) ■ ▼ Docker container (docker-compose up)
```

3.2 Cloud Target (AWS)

ECS Fargate runs the Streamlit container · **RDS PostgreSQL** replaces SQLite · **S3** stores historical data · **Kinesis** streams live ticks · **ALB** exposes a public HTTPS URL · **Terraform** provisions the entire infrastructure as code.

4. Project File Structure

```
real-time-delta-one-risk-dashboard/ ■■■ app/ ■ ■■■ dashboard.py # Interface Streamlit principale ■■■ src/ ■ ■■■ market_data.py # Ingestion données (yfinance + simulation) ■ ■■■ portfolio.py # Positions, expositions, top movers ■ ■■■ pnl.py # PnL par position, total, intraday, historique ■ ■■■ risk.py # VaR, beta, vol, drawdown, corrélation, TE ■ ■■■ database.py # Persistence SQLite / PostgreSQL ■ ■■■ utils.py # Config, logging, formatters ■■■ config/ ■ ■■■ config.yaml # Portefeuille, seuils de risque, DB ■■■ tests/ ■ ■■■ test_risk.py # 12 tests unitaires (pytest) ■■■ docs/ ■ ■■■ dashboard_screenshot.png ■■■ Dockerfile ■■■ docker-compose.yml ■■■ requirements.txt ■■■ architecture.md ■■■ ROADMAP.md ■■■ README.md
```

5. Code Walkthrough

5.1 market_data.py — Data Ingestion

Ce module est le point d'entrée des données. Il expose trois fonctions principales :

- **fetch_historical_prices(tickers, period)** — télécharge les prix de clôture ajustés via yfinance.download()
- **fetch_realtime_prices(tickers)** — récupère le dernier prix disponible pour chaque ticker
- **simulate_price_feed(tickers, base_prices, n_days)** — génère un historique synthétique via Mouvement Brownien Géométrique (fallback week-end / hors réseau)

Commande de test rapide :

```
python3 -c "import yfinance as yf; h=yf.Ticker('SPY').history(period='5d');
print(h['Close'].iloc[-1])"
```

5.2 portfolio.py — Position Management

Charge les positions depuis config.yaml, les enrichit avec les prix live, calcule les métriques de base :

- **load_positions()** — lit le YAML, retourne un DataFrame (ticker, qty, entry_price, side, geography)
- **enrich_with_live_prices()** — ajoute current_price, market_value, pnl, pnl_pct, intraday_pnl, weight_pct
- **compute_gross_exposure()** — Σ |MktValue_i| : mesure le levier total
- **compute_net_exposure()** — Σ MktValue_i : mesure le biais directionnel

5.3 pnl.py — PnL Calculation

Toutes les fonctions de calcul du PnL. La fonction clé est build_pnl_history() qui reconstitue l'historique journalier du portefeuille à partir des prix historiques.

- **compute_position_pnl()** — PnL = (P_current - P_entry) × Qty
- **build_pnl_history()** — DataFrame avec portfolio_pnl, daily_pnl, daily_return_pct
- **compute_pnl_attribution()** — contribution de chaque position au PnL total

5.4 risk.py — Risk Engine

Le coeur quantitatif du projet. Contient toutes les métriques de risque utilisées sur un desk :

- **compute_asset_beta()** — régression OLS : β = Cov(Ri, Rb) / Var(Rb)
- **compute_rolling_volatility()** — σ_ann = std(R, 20j) × $\sqrt{252}$
- **compute_var_historical()** — VaR = -Percentile(PnL, 5%) sans hypothèse de distribution
- **compute_var_parametric()** — VaR = -(μ + z × σ) avec z = 1.645 pour 95%
- **compute_drawdown_series()** — DD_t = (V_t - max(V)) / max(V) × 100
- **compute_tracking_error()** — TE = std(Rp - Rb) × $\sqrt{252}$
- **generate_risk_alerts()** — vérifie VaR, drawdown, levier, concentration vs seuils config

5.5 app/dashboard.py — Streamlit UI

7 sections affichées avec auto-refresh configurable (30s, 60s, 5min) :

Section	Contenu
Portfolio Summary (KPIs)	NAV, Total PnL, Intraday PnL, Gross/Net Exp., Levier, Beta
Risk KPIs	VaR 95%, Max Drawdown, Ann. Volatility, Tracking Error
PnL History	Courbe cumulative + barres daily PnL (Plotly, 2 sous-graphiques)
Risk Alerts	Alertes colorées si seuils dépassés (VaR, drawdown, levier, concentration)
Top Movers	N positions avec le plus grand mouvement intraday en %
Position Book	Table complète avec PnL colorisé vert/rouge
Expositions	Geographic bar chart + Asset class donut + PnL attribution

Correlation Matrix	Heatmap RdBu_r des corrélations Pearson
Drawdown Chart	Courbe du drawdown sur la période sélectionnée

6. Financial Formulas

PnL par position

```
PnL_i = (P_current_i - P_entry_i) x Qty_i
```

Gross Exposure

```
Gross = Sum( |MktValue_i| )
```

Net Exposure

```
Net = Sum( MktValue_i ) [+ = net long, - = net short]
```

Levier

```
Leverage = Gross Exposure / |NAV|
```

Beta actif

```
beta_i = Cov(R_i, R_benchmark) / Var(R_benchmark)
```

Beta portefeuille

```
beta_ptf = Sum( w_i x beta_i x sign_i )
```

Volatilité annualisée

```
sigma = std(R_daily, window=20) x sqrt(252)
```

VaR historique 95%

```
VaR = -Percentile(PnL_history, 5%)
```

VaR paramétrique 95%

```
VaR = -(mu + z_alpha x sigma) [z = 1.645]
```

Drawdown

```
DD_t = (V_t - max(V_0..V_t)) / max(V_0..V_t) x 100
```

Max Drawdown

```
MDD = min( DD_t )
```

Tracking Error

```
TE = std(R_portfolio - R_benchmark) x sqrt(252)
```

7. How to Run

7.1 Lancer le dashboard

```
# Terminal — à chaque session conda activate delta1 cd
~/Desktop/real-time-delta-one-risk-dashboard streamlit run app/dashboard.py # Ouvre
http://localhost:8501
```

7.2 Lancer les tests

```
pytest tests/ -v --cov=src # 12 tests : PnL, beta, VaR, drawdown, volatilité
```

7.3 Docker

```
docker-compose up --build # Dashboard sur http://localhost:8501 dans le conteneur
```

7.4 Push GitHub

```
git add . git commit -m "feat: description de la modification" git push # Repo :
github.com/theodoredyrlund/real-time-delta-one-risk-dashboard
```

8. Roadmap

Phase	Fonctionnalités	Statut
Phase 1 — MVP	Dashboard Streamlit, PnL, VaR, beta, drawdown, corrélation, Docker, SQLite, tests unitaires	■ Terminé
Phase 2 — Enhanced	Données intraday (1min), stress tests (-10% scenario), trade blotter, export CSV/Excel, alertes email	■ À faire
Phase 3 — Cloud	AWS ECS Fargate, RDS PostgreSQL, S3, Kinesis, Terraform IaC, CI/CD GitHub Actions → ECR → ECS	■ À faire